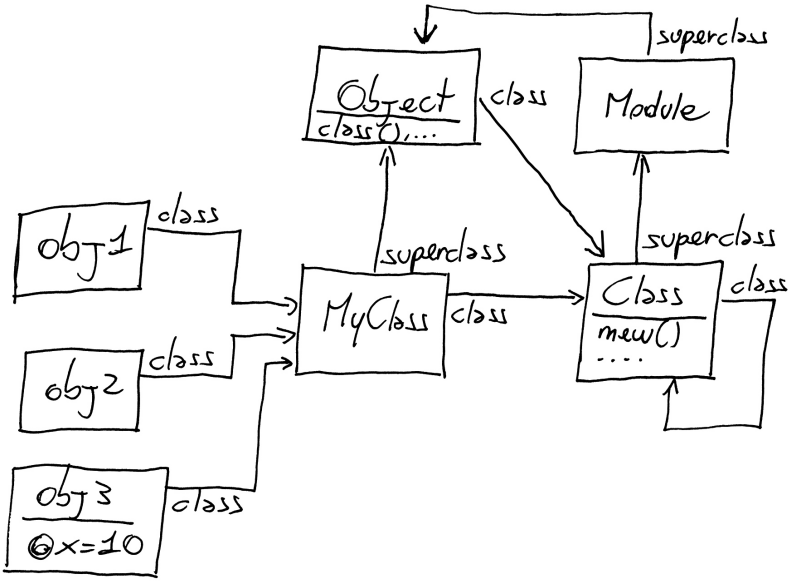


Figure 3—Bill's diagram, enhanced by you

"Stop!" Bill shouts, startling you. "This code is too complicated. To understand it, you have to learn in detail what happens when you call a method." And before you can react, he dives into yet another lecture.

When you call a method, Ruby does two things:

1. It finds the method. This is a process called *method lookup*.
2. It executes the method. To do that, Ruby needs something called `self`.

This process—find a method and then execute it—happens in every object-oriented language. In Ruby, however, you should understand the process in depth, because this knowledge will open the door to some powerful tricks. We'll talk about method lookup first, and we'll come around to `self` later.

Method Lookup

You already know about the simplest case of method lookup. Look back at Figure 1, *Instance variables live in objects; methods live in classes*, on page 18. When you call a method, Ruby looks into the object's class and finds the method there. Before you look at a more complicated example, though, you need to know about two new concepts: the receiver and the ancestors chain.